

Phase-1 Major Project Documentation & Weekly Reports

Project Title: Uncertainty-Aware Cloud Workload Forecasting and Autoscaling using HybridTimeNet

PART 1: WEEKLY WORK REPORTS (7 Weeks)

Week 1 Report: Finalized Problem Statement, Relevance, Objectives

Week No: 1 **Date:** [Insert Date] **Student Names / USN:** Samarth S Rao / [Insert USN] **Guide Name:** [Insert Guide Name] **Project Title:** Uncertainty-Aware Cloud Workload Forecasting using HybridTimeNet

Work Component (Tick Appropriately): ☒ Problem Identification / ☐ Literature Review / ☐ Design / ☐ Feasibility Study / ☐ Requirement Analysis

Work Summary: This week focused entirely on understanding the core problems in modern cloud resource provisioning. We analyzed the dual issues of over-provisioning (which leads to financial waste and increased carbon footprint) and under-provisioning (which leads to Service Level Agreement (SLA) violations and poor user experience). We identified that traditional deterministic point-forecasting models fail during high volatility. Thus, we formulated a problem statement focusing on integrating Bayesian deep learning and hybrid forecasting to quantify both aleatoric (data-driven) and epistemic (model-driven) uncertainties. We formally defined the objectives of the project.

Hours Worked: 12 Hours **Reference Links / DOIs:**

- arXiv:2303.13525 (Forecasting Workload in Cloud Computing: Towards Uncertainty-Aware Predictions)
- 10.1109/TCC.2021.3059685 (Survey on Cloud Autoscaling)

Learning / Outcomes: Successfully defined clear, actionable objectives focusing on reducing SLA violations while minimizing resource wastage using proactive autoscaling based on uncertainty bounds rather than absolute predictions.

Challenges / Gaps Identified: Translating raw workload volatility and statistical variance into actionable autoscaling metrics (number of VMs) is mathematically complex when dealing with standard deterministic predictions.

Tools Used: Google Scholar, IEEE Xplore, MS Word

Week 2 Report: Detailed Literature Survey & Gap Analysis

Week No: 2 **Date:** [Insert Date] **Student Names / USN:** Samarth S Rao / [Insert USN] **Guide Name:** [Insert Guide Name] **Project Title:** Uncertainty-Aware Cloud Workload Forecasting using HybridTimeNet

Work Component (Tick Appropriately): ☐ Problem Identification / ☒ Literature Review / ☐ Design / ☐ Feasibility Study / ☐ Requirement Analysis

Work Summary: Conducted an exhaustive literature survey focusing on cloud autoscaling, time series forecasting, and queueing theory. We reviewed standard deterministic approaches (ARIMA, standard LSTMs) and compared them with state-of-the-art probabilistic models and Reinforcement Learning (RL). We performed a gap analysis which highlighted the critical failure of existing models to account for workload stochasticity during sudden bursts. We identified that combining XGBoost, Neural Prophet, and Bayesian LSTMs (the HybridTimeNet architecture) can effectively bridge this gap.

Hours Worked: 14 Hours **Reference Links / DOIs:**

- Papers on M/M/c queueing models in cloud environments.
- Studies on Monte Carlo Dropout for Bayesian Neural Networks.

Learning / Outcomes: Gained a deep understanding of Bayesian deep learning applied to time-series forecasting. Finalized the approach of using an M/M/c queueing model to translate prediction intervals into server provisioning numbers.

Challenges / Gaps Identified: Existing solutions fail catastrophically under high volatility datasets (like the NASA or Madrid traces). A robust uncertainty quantification module is strictly necessary but computationally expensive.

Tools Used: Google Scholar, ResearchGate, Mendeley, Notion

Week 3 Report: Proposed Methodology & Expected Outcomes

Week No: 3 **Date:** [Insert Date] **Student Names / USN:** Samarth S Rao / [Insert USN] **Guide Name:** [Insert Guide Name] **Project Title:** Uncertainty-Aware Cloud Workload Forecasting using HybridTimeNet

Work Component (Tick Appropriately): ☐ Problem Identification / ☐ Literature Review / ☒ Design / ☐ Feasibility Study / ☐ Requirement Analysis

Work Summary: Formulated the core HybridTimeNet methodology. We mapped out the data ingestion pipeline, the hybrid modeling phase, the uncertainty estimation process using Monte Carlo Dropout, and the deterministic M/M/c queue-based provisioning system. We also defined the expected outcomes, heavily focusing on balancing the trade-off between the statistical accuracy of the model (RMSE/MAE) and the real-world business system metrics (SLA violations vs. overprovisioning ratio).

Hours Worked: 10 Hours **Reference Links / DOIs:** N/A

Learning / Outcomes: Established a rigorous theoretical framework for dynamically mapping predicted workloads and their associated uncertainty intervals directly to physical server counts.

Challenges / Gaps Identified: Integrating queuing theory (M/M/c) with neural network outputs requires careful scaling of metrics (e.g., converting 'requests per minute' into 'arrival rates' for the queueing model).

Tools Used: Draw.io, MS Word

Week 4 Report: System Design & Architecture

Week No: 4 **Date:** [Insert Date] **Student Names / USN:** Samarth S Rao / [Insert USN] **Guide Name:** [Insert Guide Name] **Project Title:** Uncertainty-Aware Cloud Workload Forecasting using HybridTimeNet

Work Component (Tick Appropriately): ☐ Problem Identification / ☐ Literature Review / ☒ Design / ☐ Feasibility Study / ☐ Requirement Analysis

Work Summary: Architected the high-level system design. We divided the monolithic concept into a robust microservices architecture. Created comprehensive Data Flow Diagrams (DFDs), UML Class diagrams, and Sequence Diagrams to illustrate the precise data flow from the Python-based Workload Predictor, into the Golang-based Orchestrator, and finally to the Cloud Simulator and React Dashboard.

Hours Worked: 15 Hours **Reference Links / DOIs:** N/A

Learning / Outcomes: Finalized the microservices-based architecture communicating via gRPC/REST. Designed the feedback loop that allows the simulator to report SLA metrics back to the dashboard for live visualization.

Challenges / Gaps Identified: Designing real-time data flow pipelines requires careful synchronization; the simulator must wait for the orchestrator to provision servers before routing requests to avoid false SLA violations due to race conditions.

Tools Used: Draw.io, Lucidchart, Figma

Week 5 Report: Technical Requirements & Constraints

Week No: 5 **Date:** [Insert Date] **Student Names / USN:** Samarth S Rao / [Insert USN] **Guide Name:** [Insert Guide Name] **Project Title:** Uncertainty-Aware Cloud Workload Forecasting using HybridTimeNet

Work Component (Tick Appropriately): ☐ Problem Identification / ☐ Literature Review / ☐ Design / ☐ Feasibility Study / ☒ Requirement Analysis

Work Summary: Documented all exact software and hardware requirements necessary to build and run the simulation. Finalized the tech stack: Python (PyTorch/Scikit-learn) for the ML model, Golang for the high-concurrency orchestrator, and React/Next.js for the visual dashboard. We also analyzed and documented system constraints, specifically the inherent "cold start" delay in VM provisioning and the computational overhead of running Monte Carlo Dropout at inference time.

Hours Worked: 8 Hours **Reference Links / DOIs:** N/A

Learning / Outcomes: Prepared a comprehensive, production-ready list of technical dependencies. Developed constraint mitigation strategies (such as optimizing the number of forward passes in MC Dropout).

Challenges / Gaps Identified: Managing the computational resource overhead of the prediction system itself; if the autoscaler uses too much CPU, it defeats the purpose of optimization.

Tools Used: MS Word, GitHub (for repository planning)

Week 6 Report: Project Planning, Feasibility Analysis & Timeline

Week No: 6 **Date:** [Insert Date] **Student Names / USN:** Samarth S Rao / [Insert USN] **Guide Name:** [Insert Guide Name] **Project Title:** Uncertainty-Aware Cloud Workload Forecasting using HybridTimeNet

Work Component (Tick Appropriately): ☐ Problem Identification / ☐ Literature Review / ☐ Design / ☒ Feasibility Study / ☐ Requirement Analysis

Work Summary: Developed a detailed project timeline utilizing a Gantt chart layout spanning 14 weeks. Conducted thorough feasibility studies: Operational, Technical, and Economic. We broke down the upcoming development cycle into structured phases (Model Training, Orchestrator Setup, Simulator Build, Integration).

Hours Worked: 9 Hours **Reference Links / DOIs:** N/A

Learning / Outcomes: Confirmed that the project is highly feasible within the academic timeframe and available computational resources. Established clear, measurable milestones for the Phase 2 development cycle.

Challenges / Gaps Identified: Anticipating strict time constraints regarding the training and hyperparameter tuning of large deep learning ensemble models.

Tools Used: MS Excel, MS Project, Jira (for task tracking)

Week 7 Report: Presentation & Documentation

Week No: 7 **Date:** [Insert Date] **Student Names / USN:** Samarth S Rao / [Insert USN] **Guide Name:** [Insert Guide Name] **Project Title:** Uncertainty-Aware Cloud Workload Forecasting using HybridTimeNet

Work Component (Tick Appropriately): ☒ Problem Identification / ☒ Literature Review / ☒ Design / ☒ Feasibility Study / ☒ Requirement Analysis

Work Summary: Compiled, reviewed, and finalized all research, architectural designs, and planning matrices from the previous six weeks into the official 12-chapter Major Project Phase-1 documentation. Created highly visual presentation slides for the upcoming Phase-1 academic evaluation, ensuring all institutional formatting guidelines were strictly met.

Hours Worked: 14 Hours **Reference Links / DOIs:** N/A

Learning / Outcomes: Successfully synthesized deep technical research and system design into a cohesive, structured academic documentation format ready for faculty review.

Challenges / Gaps Identified: Distilling complex Bayesian deep learning concepts into digestible presentation slides for a general computer science audience.

Tools Used: LaTeX / Overleaf, MS Word, Canva, MS PowerPoint

PART 2: DETAILED 12-CHAPTER DOCUMENTATION

1. Abstract

1.1 Problem Overview: Modern cloud computing environments process billions of requests daily, resulting in highly dynamic and stochastic workload patterns. Traditional proactive autoscaling systems predominantly rely on deterministic point-forecasting models (such as ARIMA or standard LSTMs). These models predict a single future value without providing any measure of confidence. When exposed to high volatility or sudden bursts in traffic, these point forecasts fail. This leads to two critical problems: under-provisioning, which causes catastrophic Service Level Agreement (SLA) violations and degrades user experience, or over-provisioning, which results in significant financial waste and an increased carbon footprint.

1.2 Proposed Solution: To address this, we propose *HybridTimeNet*, an uncertainty-aware predictive autoscaling architecture. Rather than outputting a single prediction, HybridTimeNet utilizes Bayesian deep learning—specifically LSTMs equipped with Monte Carlo Dropout—in an ensemble with Neural Prophet and XGBoost. This approach generates a probabilistic workload forecast, effectively quantifying both aleatoric (inherent data noise) and epistemic (model ignorance) uncertainties. These quantified uncertainty bounds (upper and lower prediction intervals) are then fed into an M/M/c queueing theory model. This mathematical model translates the probabilistic load bounds into robust, risk-averse server provisioning decisions.

1.3 Expected Outcome: The implementation of this architecture is expected to yield a highly resilient cloud autoscaling framework. By provisioning resources against the upper bounds of uncertainty during volatile periods, the system is expected to significantly reduce SLA violation rates (by an estimated 15-20% compared to deterministic baselines) while maintaining tight server utilization rates during stable periods, effectively avoiding unnecessary over-provisioning.

2. Introduction

2.1 Background: Cloud elasticity—the ability to dynamically provision and de-provision computing resources—is the fundamental advantage of cloud computing. Reactive autoscaling triggers scaling actions only after a threshold is breached, inherently causing a delay known as a "cold start." Proactive autoscaling attempts to circumvent this by

predicting future load and scaling servers before the demand spike occurs. However, the efficacy of proactive scaling is entirely bottlenecked by the accuracy and reliability of the forecasting model.

2.2 Need for the Project: Existing forecasting models treat their predictions as absolute certainty. When these deterministic forecasts inevitably drift during unexpected traffic bursts (such as a viral event or a sudden sale), the autoscaler makes incorrect decisions, and the system crashes. By understanding the *confidence* (uncertainty) of a prediction, the orchestration system can provision a safe computing buffer dynamically. If the model is unsure about the future, it provisions more conservatively to prevent crashes; if it is highly confident, it provisions tightly to save costs.

2.3 Scope: The scope of this project encompasses the end-to-end development of the HybridTimeNet predictive model, an intelligent orchestrator microservice, a highly concurrent cloud load simulator to mock real-world web traffic and VM provisioning, and a real-time web dashboard for visualization. The scope is restricted to scaling CPU and request-handling metrics, utilizing open-source real-world HTTP trace datasets (e.g., NASA, Calgary, ClarkNet).

3. Problem Statement, Relevance & Objectives

3.1 Problem Statement: To design, develop, and evaluate an uncertainty-aware cloud workload forecasting and autoscaling system that explicitly quantifies aleatoric and epistemic uncertainties to optimize proactive resource allocation, aiming to strictly minimize SLA violations under high workload volatility without excessive resource overallocation.

3.2 Importance of the Project: With the exponential growth of cloud-native applications and microservices, optimizing resource allocation directly translates to millions of dollars in cost savings for cloud providers and enterprise consumers. Furthermore, minimizing idle, over-provisioned servers drastically reduces the power consumption and carbon footprint of massive data centers. Ensuring high availability prevents revenue loss during critical traffic spikes.

3.3 Objectives:

1. To design and train the HybridTimeNet ensemble model utilizing PyTorch.
2. To implement robust uncertainty quantification using Bayesian techniques (Monte Carlo Dropout).
3. To integrate an M/M/c queueing theory component capable of translating request rates and uncertainty bounds into physical server counts.
4. To build a robust, Golang-based simulated environment to rigorously benchmark the proposed architecture against baseline deterministic models.
5. To visualize system performance, SLA metrics, and financial costs via a real-time React dashboard.

4. Literature Survey & Gap Analysis

4.1 Existing Systems/Research: Extensive literature exists on cloud autoscaling. Early approaches relied heavily on statistical time-series models like ARIMA, SARIMA, and Exponential Smoothing. The advent of Deep Learning saw a shift towards Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) for workload point forecasting. Most recently, research has heavily trended towards Reinforcement Learning (RL) and Q-Learning for policy-based autoscaling.

4.2 Limitations of Existing Solutions: Traditional time series models fail to capture the complex, non-linear, and long-term dependencies present in modern HTTP cloud traces. While standard deep learning models (LSTMs) capture non-linearities, they are strictly deterministic—they cannot express "doubt" when facing novel, out-of-distribution data patterns. RL models, while promising, suffer from massive sample inefficiency, require extensive training in production environments, and exhibit highly unstable, unpredictable behavior during initial exploration phases, making them dangerous for critical infrastructure.

4.3 Gap Identified: There is a severe lack of applied probabilistic forecasting in production cloud control planes. While Bayesian Deep Learning exists in academia, its application to direct hardware provisioning via queueing theory is largely unexplored. There is a critical research gap for a hybrid architecture that leverages the speed of modern ensemble models (XGBoost/Prophet) alongside the uncertainty bounds of Bayesian Neural Networks to make risk-averse, mathematically sound scaling decisions.

5. Proposed Methodology

5.1 Working of the Proposed System: The proposed architecture operates in a continuous, cyclic pipeline:

- 1. Data Ingestion & Preprocessing:** Historical workload traces are ingested, normalized, and engineered into sliding window sequences.
- 2. Hybrid Forecasting Engine:** The data is fed into HybridTimeNet. The Bayesian LSTM performs multiple forward passes (with dropout active) to generate a distribution of predictions. Neural Prophet handles seasonality, and XGBoost handles residual non-linearities. The output is a predicted mean workload and a variance (uncertainty interval).
- 3. Decision Engine (Orchestrator):** The Orchestrator receives the predicted load and its upper uncertainty bound. It inputs these values into an M/M/c (Markovian arrival, Markovian service, c servers) queueing formula. It calculates the exact number of servers (c) required to ensure that the probability of a request waiting longer than the SLA threshold is minimized.
- 4. Actuation:** The Orchestrator commands the Cloud Simulator to spin up or tear down mock virtual machines.

5.2 Technologies/Tools Used:

- **Machine Learning:** Python 3.10+, PyTorch, Scikit-learn, XGBoost, Neural Prophet.
- **Orchestration & Simulation:** Golang 1.21+ (chosen for superior concurrency and goroutine performance to simulate thousands of requests).
- **Frontend / Visualization:** JavaScript, React, Next.js, Recharts.
- **Infrastructure:** Docker, Docker Compose, gRPC, REST APIs.

5.3 Workflow/Process: Raw Trace Data -> Feature Engineering -> HybridTimeNet Inference -> Mean & Variance Extraction -> Upper Bound Calculation -> M/M/c Queue Solver -> Provisioning Command -> Simulator Update -> Metrics Feedback Loop -> Dashboard Visualization.

6. System Design & Architecture

6.1 Architecture Diagram: *(Note: Visual diagrams to be inserted in the final print document. The architecture features a central event bus connecting three core microservices.)*

1. **Load Predictor Service (Python):** Exposes a gRPC/REST endpoint. Loads pre-trained PyTorch/XGBoost models into memory. Accepts current workload windows and returns future predictions with confidence intervals.
2. **Orchestrator Service (Go):** The brain of the system. Contains the M/M/c logic, stores SLA configuration parameters, and triggers scaling rules.
3. **Cloud Simulator (Go):** Generates HTTP-like traffic based on the historical datasets. Routes traffic to a pool of "mock" worker instances. Calculates real-time latency and tracks dropped/delayed requests (SLA violations).

6.2 Modules/Components:

- **Data Preprocessor:** Handles missing values and Min-Max scaling.
- **Bayesian LSTM Module:** Implements MC Dropout.
- **Queueing Solver:** Numerically solves Erlang-C formulas to find optimal server counts.
- **Metrics Aggregator:** Collects live telemetry from the simulator for the dashboard.

6.3 Database Design / DFD / UML Diagrams: The system relies on in-memory data structures (like Redis or native Go maps) for extreme speed, rather than traditional relational databases. DFDs illustrate a circular feedback loop where the Simulator's output latency continuously informs the user dashboard, while the Predictor operates on a strict look-ahead schedule (e.g., predicting 5 minutes into the future).

7. Technical Requirements & Constraints

7.1 Hardware Requirements:

- **Development/Training:** Minimum 16GB RAM, Modern Multi-core CPU (Intel i7/Ryzen 7), Dedicated GPU (Nvidia RTX 3060+ with CUDA support) highly recommended for LSTM training.
- **Deployment/Simulation:** Minimum 8GB RAM, Multi-core CPU to handle Golang concurrency.

7.2 Software Requirements:

- Operating System: Linux (Ubuntu 22.04 LTS) or Windows 11 with WSL2.
- Containerization: Docker Desktop / Docker Compose.
- Runtimes: Python 3.10+, Go 1.21+, Node.js 18+.

7.3 Constraints and Limitations:

- **Simulation Fidelity:** A local Golang simulator cannot perfectly replicate the complex network latency, hypervisor overhead, and disk I/O bottlenecks present in real-world environments like AWS EC2 or GCP Compute Engine.
 - **Inference Latency:** Running Monte Carlo Dropout requires 50-100 forward passes of the neural network. This inference must be heavily optimized so the prediction generation does not become slower than the autoscaling window itself.
 - **Cold Start Assumption:** The system currently simulates a fixed cold-start penalty (e.g., 30 seconds for a VM to boot), whereas real-world cold starts are highly variable.
-

8. Project Planning & Timeline

8.1 Development Phases:

- **Phase 1 (Weeks 1-4):** Problem definition, extensive literature review, architectural design, and technology stack finalization. (Completed)
- **Phase 2 (Weeks 5-8):** Data engineering, implementation of the HybridTimeNet architecture, Bayesian LSTM training, and hyperparameter tuning.
- **Phase 3 (Weeks 9-11):** Development of the Golang Orchestrator and the concurrent Cloud Simulator. Integration of the M/M/c queueing logic.
- **Phase 4 (Weeks 12-14):** End-to-end integration via Docker Compose, API bridging, React Dashboard development, and comprehensive benchmarking against baseline models.

8.2 Feasibility Analysis:

- *Technical Feasibility:* High. The required ML libraries (PyTorch) and high-performance languages (Go) are mature, open-source, and heavily documented.
- *Operational Feasibility:* High. The project successfully abstracts complex mathematical models into an automated orchestration pipeline, requiring minimal human intervention once deployed.
- *Economic Feasibility:* High. Relies entirely on free, open-source software and local compute resources. No enterprise cloud billing is required for the simulation phase.

8.3 Timeline / Gantt Chart: *(A detailed Gantt chart visualizing the 14-week concurrent development process of the Python backend and Golang simulator will be attached in the final submission annexure.)*

9. Expected Outcomes

9.1 Expected Results: We expect to deliver a fully functional, containerized autoscaling simulation framework. Upon benchmarking with highly volatile datasets (e.g., the NASA HTTP traces or the WIDE project traces), we expect the uncertainty-aware HybridTimeNet to proactively provision resources effectively enough to reduce SLA violations by at least 15% to 20% compared to a purely deterministic LSTM baseline.

9.2 Benefits of the Project:

- **Reliability:** Drastically improves the robustness of cloud-native applications during unprecedented traffic spikes.
 - **Cost Efficiency:** By utilizing uncertainty bounds to scale down aggressively during highly predictable, low-variance periods, the system minimizes server uptime.
 - **Academic Contribution:** Provides a tangible, implementable framework that bridges the gap between theoretical Bayesian deep learning and practical, queueing-theory-based cloud infrastructure management.
-

10. Implementation Details (Optional for Phase 1)

As this is the Phase 1 report, full implementation is pending. However, initial implementation details have been drafted:

- **ML Pipeline:** The `hybrid_time_net/` directory has been scaffolded. `train.py` will handle the PyTorch training loop. The LSTM class uses standard PyTorch `nn.LSTM` modules but strictly maintains `dropout=0.2` during the `.eval()` inference phase to enable Monte Carlo sampling.
 - **Simulator:** The `services/orchestrator/` directory contains `main.go`. We use Go channels and goroutines to simulate thousands of concurrent HTTP requests arriving at the system, calculating latency by tracking timestamps across channel buffers.
-

11. Conclusion

11.1 Summary: The Phase-1 lifecycle of the major project has successfully established the core theoretical foundation, problem scope, and concrete architectural design for an uncertainty-aware predictive autoscaler. By shifting the paradigm from absolute point-forecasts to probabilistic forecasting, the proposed HybridTimeNet system is intelligently designed to handle the stochastic, chaotic nature of real-world cloud workloads safely and efficiently.

11.2 Future Improvements: While the current scope utilizes a custom Golang simulator for validation, future iterations of this project (Phase 3 or Post-Graduation) could involve writing a Custom Controller (Custom Resource Definition - CRD) for Kubernetes (K8s). This would allow HybridTimeNet to directly control actual Docker containers (Pods) on a live K3s or Minikube cluster. Additionally, exploring Reinforcement Learning to dynamically tune the SLA confidence interval based on live user feedback presents an exciting future research avenue.

12. References

12.1 Research Papers:

1. R. Qiao et al., "Forecasting Workload in Cloud Computing: Towards Uncertainty-Aware Predictions and Transfer Learning," arXiv:2303.13525, 2023.
2. L. Llorido-Botran et al., "A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments," Journal of Grid Computing, 2014.
3. Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning," ICML, 2016.

12.2 Websites / Documentation:

1. PyTorch Documentation - <https://pytorch.org/docs/> (<https://pytorch.org/docs/>).
2. Go Concurrency Patterns - <https://go.dev/blog/pipelines> (<https://go.dev/blog/pipelines>).
3. Kubernetes Horizontal Pod Autoscaling - <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/> (<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>).

12.3 Other Sources Used:

1. **Datasets:** NASA HTTP Traces, Calgary HTTP Traces, ClarkNet Traces (Available via the Internet Traffic Archive).
2. **Tools:** Draw.io for architecture diagrams, LaTeX for typesetting.